Stack & It's Applications

CSE 4303: Data Structures
Lecture 3

Asaduzzaman Herok
Department of CSE
Islamic University of Technology

November 24, 2025

Agenda

1. Introduction to Stacks
2. The Stack ADT
3. Array Implementation
4. Linked Implementation
5. Auxiliary Operations
6. Comparison
7. Reversing a Data Series
8. Parentheses Checker
9. Expression Notations
10. Infix to Postfix
11. Evaluating Postfix
12. Prefix Operations
13. Recursion
14. End

What is a Stack?

Stack

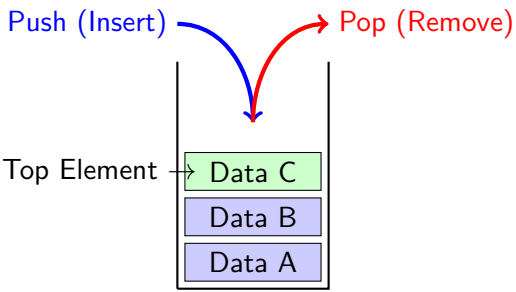
A Stack is a linear data structure that follows a specific order for insertion and deletion.

The Rule: LIFO

- **L**ast **I**n **F**irst **O**ut.
- The element inserted last is the first one to be removed.

Real World Analogies:

- A stack of cafeteria trays.
- A stack of books on a desk.
- The "Undo" button in text editors.



The Stack Operations

As an Abstract Data Type (ADT), a Stack defines the following interface:

Primary Operations

- **Push(x):** Adds element x to the top of the stack.
- **Pop():** Removes the top element from the stack.

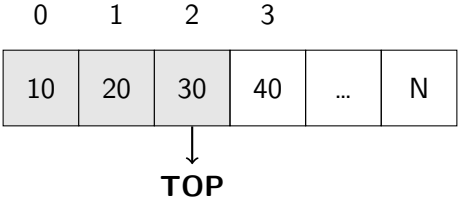
Auxiliary Operations

- **Peek() / Top():** Returns the top element without removing it.
- **isEmpty():** Returns true if the stack contains no elements.
- **isFull():** Returns true if the stack cannot accept more elements (implementation specific).

Implementation 1: Array-Based

We can use a simple array and a variable `TOP` to track the last inserted element.

- 👉 **MAX_SIZE:** The fixed capacity of the stack.
- 👉 **TOP:** Index of the topmost element.
- 👉 **Initialization:** `TOP = -1` (Empty Stack).



Array Operation: PUSH & POP

Operation: Push

Algorithm: Add element to index `TOP + 1`.
Corner Case: Stack Overflow (Array is full).

Push(item)

```
IF TOP == MAX_SIZE - 1
  Print "OVERFLOW"
  Return
ELSE
  TOP = TOP + 1
  STACK[TOP] = item
```

Operation: Pop

Algorithm: Return element at `TOP` and decrement `TOP`.
Corner Case: Stack Underflow (Empty)

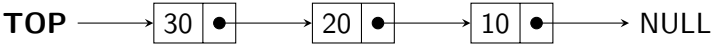
Pop()

```
IF TOP == -1
  Print "UNDERFLOW"
  Return NULL
ELSE
  DATA = STACK[TOP]
  TOP = TOP - 1
  Return DATA
```

Implementation 2: Linked List-Based

To avoid fixed size limitations, we use a Linked List.

- 👉 The **HEAD** of the list acts as the **TOP** of the Stack.
- 👉 **Push:** Insert at Head ($O(1)$).
- 👉 **Pop:** Delete at Head ($O(1)$).



Linked List Operations

Push(item)

```
NewNode = malloc()
NewNode->Data = item
NewNode->Next = TOP
TOP = NewNode
```

Pop()

```
IF TOP == NULL
  Error "Underflow"
Temp = TOP
TOP = TOP->Next
FREE Temp
```

Auxiliary Operation: PEEK (Top)

Definition: Returns the value of the top element *without* removing it. This is useful for checking the current state before deciding to pop.

Array Implementation

Peek()

```
IF TOP == -1
  Return ERROR (Empty)
ELSE
  Return STACK[TOP]
```

Linked Implementation

Peek()

```
IF TOP == NULL
  Return ERROR (Empty)
ELSE
  Return TOP->Data
```

Auxiliary Operation: isEmpty

Definition: Checks if the stack contains no elements.
Usage: Always call this before `Pop()` to prevent **Stack Underflow**.

- 👉 **Array Condition:** `TOP == -1`
- 👉 **Linked List Condition:** `TOP == NULL`

Algorithm

```
IF TOP == -1 // or TOP == NULL
  Return TRUE
ELSE
  Return FALSE
```

Auxiliary Operation: isFull

Definition: Checks if the stack has reached its maximum capacity.
Usage: Always call this before `Push()` to prevent **Stack Overflow**.

Important Distinction

- 👉 **Array Stack:** Returns `true` if the array is full.
- 👉 **Linked Stack:** Generally always returns `false` (unless system memory is exhausted).

Array Algorithm

```
IF TOP == MAX_SIZE - 1
  Return TRUE
ELSE
  Return FALSE
```

Comparison: Array vs Linked List

Feature	Array Stack	Linked Stack
Size	Fixed (Static)	Dynamic (Grows/Shrinks)
Efficiency	$O(1)$ (Very Fast)	$O(1)$ (Pointer overhead)
Memory	No pointer overhead	Extra memory for pointers
Limits	Stack Overflow likely	Limit is System RAM

Key Takeaway: Use Array implementation if you know the maximum size in advance. Use Linked implementation for flexibility.

Application 1: Reversing a List

Concept: The LIFO (Last-In, First-Out) property of a stack naturally reverses any sequence of data.

Algorithm:

1. Read elements from the source (Array, Linked List, String) and **Push** them onto the stack one by one.
2. Once all are pushed, **Pop** elements one by one until the stack is empty.
3. Store/Print the popped elements.

Example: String Reversal

Input: "HELLO"
Stack State: (Bottom) H $\rightarrow$ E $\rightarrow$ L $\rightarrow$ L $\rightarrow$ O (Top)
Pop Order: O, L, L, E, H

Application 2: Parentheses Balancing

Problem: Check if an expression has balanced delimiters: $(), \{\}, []$.

Algorithm:

1. Scan string from left to right.
2. If char is Opening `(`, `{`, `[`, **Push** onto stack.
3. If char is Closing `)`, `}`, `]`:
   - If Stack is Empty $\rightarrow$ **Error** (Unbalanced).
   - Else, **Pop** top. If Popped char doesn't match the closing type $\rightarrow$ **Error**.
4. After scanning, if Stack is NOT Empty $\rightarrow$ **Error** (Missing closing bracket).

Parentheses Checker Trace

Expression: `{ a + ( b * c ) }`

Scan	Action	Stack Content
{	Push	{
a	Ignore	{
+	Ignore	{
(	Push	{ (
...	...	...
)	Pop	{ *(Matched '(' with ')')*
}	Pop	Empty *(Matched '{' with '}')*
End	Check	Empty → **Valid**

Infix, Postfix, and Prefix

- **Infix:** Operator between operands. Human readable.
  Example: $A + B$
- **Postfix (Reverse Polish):** Operator after operands. Machine friendly (No parenthesis needed).
  Example: $AB+$
- **Prefix (Polish):** Operator before operands.
  Example: $+AB$

Order of Precedence (High to Low):

1. Exponentiation ($\wedge$),
2. Modulas
3. Multiplication ($*$) / Division ($/$)
4. Addition ($+$) / Subtraction ($-$)

Deep Dive: Expression Notations

1. Infix (Human Standard)

- 👉 Operator **between** operands.
- 👉 Relies on *Precedence* and *Parentheses* to avoid ambiguity.
- 👉 Example: $A + B * C$

2. Prefix (Polish)

- 👉 Operator **before** operands.
- 👉 No parentheses required.
- 👉 Example: $+ \ A \ * \ B \ C$

3. Postfix (Reverse Polish)

- 👉 Operator **after** operands.
- 👉 Used by compilers and calculators (HP).
- 👉 Perfectly suited for **Stack** evaluation.
- 👉 Example: $A \ B \ C \ * \ +$

Operator Hierarchy: Precedence & Associativity

When converting Infix expressions, two rules decide the order of operations:

1. **Precedence:** Which operator wins when they are *different*?
2. **Associativity:** Which operator wins when they are *the same*?

Precedence	Operator	Associativity
1 (Highest)	Parentheses ( )	-
2	Exponentiation ($\wedge$)	**Right to Left**
3	Multi ($*$), Div ($/$), Mod ($\%$)	**Left to Right**
4 (Lowest)	Add ($+$), Sub ($-$)	**Left to Right**

Application 3: Infix to Postfix Algorithm

```
1: Add ")" to the end of the infix expression
2: Push "(" on to the stack
3: Repeat until each character in the infix notation is scanned
    IF a "(" is encountered, push it on the stack
    IF an operand is encountered, add it to the postfix expression.
    IF a ")" is encountered, then
      a. Repeatedly pop from stack and add it to the postfix expression until a
          "(" is encountered.
      b. Discard the "(" from stack and do not add it to the postfix expression
    IF an operator X is encountered, then
      a. Repeatedly pop from stack and add each poped operator to the postfix
          expression having same precedence or a higher precedence than X
      b. Push the operator to the stack
    [END OF IF]
4: Repeatedly pop from the stack and add it to the postfix expression.
5: EXIT
```

Trace: Infix to Postfix

Expression: $A + B * C$

Symbol	Stack	Output
A	Empty	A
+	+	A
B	+	A B
*	+ *	A B
C	+ *	A B C
End	Empty	A B C * +

Logic Note: '*' was pushed because it has higher precedence than '+'. At the end, we popped '*' then '+'.

Class Excercise

$$A - (B \,/\, C \;+\; (\; D \,\%\, E \;*\; F \;) \,/\, G) \;*\; H$$

Impact of Associativity on Conversion

Left-to-Right Associativity ($+, -, *, /$)

- Interpretation: $A - B - C \implies (A - B) - C$
- **Stack Rule:** If incoming operator has **equal** precedence to Top of Stack, **POP** the stack.
- Result: $A\ B - C -$

Right-to-Left Associativity ($\wedge$)

- Interpretation: $A \wedge B \wedge C \implies A \wedge (B \wedge C)$
- **Stack Rule:** If incoming operator has **equal** precedence to Top of Stack, **PUSH** (do not pop).
- Result: $A\ B\ C \wedge \wedge$

Application 4: Evaluating Postfix

Computers prefer Postfix because it eliminates operator precedence rules during calculation.

Algorithm:

```
1: Add a ")" at the end of the postfix expression
2: Scan every character of the postfix expression and repeat Steps 3 and
    4 until ")"is encountered
3: IF an operand is encountered, push it on the stack
   IF an operator X is encountered, then
        a. Pop the top two elements from the stack as A and B as A and B
        b. Evaluate B X A, where A is the topmost element and B is the
           element below A.
        c. Push the result of evaluation on the stack
    [END OF IF]
4: SET RESULT equal to the topmost element of the stack
5: EXIT
```

Trace: Postfix Evaluation

Expression: 5 3 + 2 * (Equivalent to $(5+3)*2$)

Push 5	Push 3	Op + (5+3)	Push 2	Op * (8*2)
5	3 5	8	2 8	16

Application 5: Infix to Prefix

Strategy: We can reuse the Infix $\rightarrow$ Postfix logic.
Algorithm:

1. **Reverse** the Infix string.
2. Swap brackets: Change '(' to ')' and ')' to '('.
3. Convert this Modified String to **Postfix** (using App 3 logic).
4. **Reverse** the resulting Postfix string.

Example: $A + B$

👉 Reverse: $B + A$

👉 To Postfix: $BA+$

👉 Reverse: $+AB$ (Result)

Application 6: Evaluating Prefix

Similar to Postfix, but we scan from **Right to Left**.

Algorithm:

1. Scan expression **Right to Left**.
2. If **Operand**: Push to Stack.
3. If **Operator**:
   - Pop Top element ($\rightarrow$ Op1). **Note order difference**
   - Pop Next element ($\rightarrow$ Op2).
   - Calculate: `Result = Op1 Operator Op2`.
   - Push `Result`.

Example: $+\ 4\ 2$

👉 Push 2, Push 4.

👉 Op $+$: Pop 4 (Op1), Pop 2 (Op2). Calc $4 + 2$. Push 6.

Application 7: Recursion

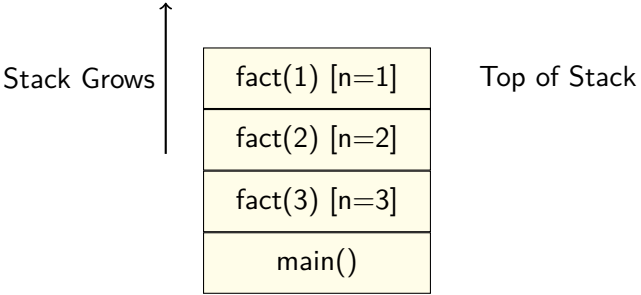
Definition: A function calling itself.

How it works (The System Stack):

- 👉 Every time a function is called, an **Activation Record** (or Stack Frame) is pushed onto the system stack.
- 👉 This record contains:
  - Local variables.
  - Return address (where to go back to).
  - Parameters.
- 👉 When the function returns, the frame is **Popped**.

Visualizing Recursion Stack

Code: `fact(3)` calls `fact(2)` calls `fact(1)`



When fact(1) returns, it is popped, and control returns to fact(2).

🤝 Reference & Q&A

Reference: Data Structures c Using 2nd Edition, Reema Thareja

👉 **Chapter 7**: 7.1, 7.2, 7.3, 7.4, 7.5, 7.7 (Covered topics only)

Thank You!

👏 Thank you for your time and attention. 👏

🗨 Any questions or clarifications. 💭